

Sistemas Paralelos

Autor: Tomás Blanco

Cátedra: Sistemas Paralelos – Federico Gonzales – UNTDF

1. Abstract

Este trabajo evalúa empíricamente el impacto de diferentes modelos de concurrencia y paralelismo en Python para la resolución de tareas intensivas en CPU (CPU-bound). Específicamente, se aborda el problema de calcular el factorial de una lista de enteros grandes mediante el particionado de datos y procesamiento en paralelo, finalizando con una reducción secuencial para obtener el promedio de los resultados. Para el análisis, se implementaron cinco variantes utilizando exclusivamente la biblioteca estándar de CPython: un modelo secuencial base y cuatro arquitecturas concurrentes empleando threading, multiprocessing, ThreadPoolExecutor y ProcessPoolExecutor. Se registraron y compararon los tiempos de ejecución y la escalabilidad de cada enfoque al variar la cantidad de workers. A través de esta experimentación, el trabajo expone las limitaciones de rendimiento impuestas por el Global Interpreter Lock (GIL) en el manejo de hilos para cálculos matemáticos, demostrando empíricamente por qué la paralelización basada en procesos constituye la estrategia óptima para este tipo de cargas de trabajo en un único nodo.

2. Introducción

En este trabajo se exploró la eficiencia de la computación paralela y concurrente en Python para la resolución de tareas intensivas en CPU (CPU-bound). Para ello, se desarrolló un motor de benchmarking automatizado capaz de evaluar el rendimiento computacional al procesar datos, específicamente aplicado al cálculo de factoriales. Se contrastó el rendimiento de una ejecución secuencial base contra cuatro arquitecturas provistas por la biblioteca estándar de Python: manejo manual de hilos (threading), pool de hilos (ThreadPoolExecutor), procesos nativos (multiprocessing) y pool de procesos (ProcessPoolExecutor). El objetivo principal fue registrar los tiempos de ejecución y calcular el *speedup* y la eficiencia al variar la cantidad de *workers*. Esto permitió analizar empíricamente cómo el *Global Interpreter Lock* (GIL) de CPython penaliza a los hilos en operaciones matemáticas, y por qué el multiprocessing resulta ser el enfoque adecuado.

3. Metodología

El código se encuentra dividido en una parte cambiante y preparada para atacar diferentes problemas planteados a lo largo de los trabajos prácticos.

En la Raíz se encuentran las estrategias de paralización y concurrencia (secuencial, threads o process) y también un controlador llamado benchmark que gestiona las acciones pertinentes dado un argumento ingresado (archivo de origen, titulo, estrategia, numero de workers, archivo de datos, numero de datos, datos por argumento, archivo de preprocesado de datos, archivos resultados, etc.)

En las carpetas divididas por trabajo practico se encuentra el módulo para atacar el problema específico.

- **División de datos:** Explica cómo funciona tu función preparar_datos (el particionado en *chunks*). De manera resumida divide un arreglo de números en pequeños arreglos o chunks, la cantidad de arreglos es proporcional a la cantidad de workers. Esto tiene el propósito de repartir un problema grande en problemas pequeños y más manejables.
- **Procesamiento:** Cada worker recibirá su chunk y resolverá la factorial de cada elemento en él. Una vez terminada la tarea entregará el chunk y lo devolverá.
- **Reducción:** Cómo se combinan los resultados parciales para llegar al resultado final. Cuando todos los workers terminen, de una estrategia secuencia se aplanará la lista de listas (lista de chunks) y se realizara el promedio.

4. Implementación

[Pega aquí el link a tu GitHub]

5. Experimentos

Información Previa:

- *Hardware:* 2,7Hz Intel i5 de 2 Cores
- *Entrada:* Arreglo ->
 - *Escenario A:* [100, 120, 140, 160, 180, 200, 220, 240]
 - *Escenario B:* 10^4 números aleatorios entre 5000 y 10.000
- *Salida:* Promedio, tiempo de cómputo, speedup, eficiencia, números de workers y estrategia. Formato tabla csv

Escenario A:

Tipo	Workers	Tiempo (s)	Speedup	Eficiencia (%)
------	---------	------------	---------	----------------

Secuencial (Base)	1	0.000043	1.00x	100.0%
Threads	1	0.000101	0.43x	42.8%
Threads	2	0.000778	0.06x	2.8%
Threads	3	0.000165	0.26x	8.8%
Threads	4	0.000141	0.31x	7.7%
Multithreads (Pool)	1	0.000220	0.20x	19.7%
Multithreads (Pool)	2	0.000437	0.10x	5.0%
Multithreads (Pool)	3	0.000564	0.08x	2.6%
Multithreads (Pool)	4	0.000622	0.07x	1.7%
Process	1	0.113556	0.00x	0.0%
Process	2	0.098289	0.00x	0.0%
Process	3	0.129420	0.00x	0.0%
Process	4	0.215189	0.00x	0.0%

Multiprocess (Pool)	1	0.086197	0.00x	0.1%
Multiprocess (Pool)	2	0.102947	0.00x	0.0%
Multiprocess (Pool)	3	0.100214	0.00x	0.0%
Multiprocess (Pool)	4	0.124174	0.00x	0.0%

Escenario B:

Tipo	Workers	Tiempo (s)	Speedup	Eficiencia (%)
Secuencial (Base)	1	23.8853	1.00x	100.0%
Threads	1	22.6805	1.05x	105.3%
Threads	2	22.5589	1.06x	52.9%
Threads	3	22.5514	1.06x	35.3%
Threads	4	22.7331	1.05x	26.3%
Multithreads (Pool)	1	22.6098	1.06x	105.6%
Multithreads (Pool)	2	22.8182	1.05x	52.3%
Multithreads (Pool)	3	22.6740	1.05x	35.1%

Multithreads (Pool)	4	22.8598	1.04x	26.1%
Process	1	24.0286	0.99x	99.4%
Process	2	15.7190	1.52x	76.0%
Process	3	12.4732	1.91x	63.8%
Process	4	12.0700	1.98x	49.5%
Multiprocess (Pool)	1	24.1729	0.99x	98.8%
Multiprocess (Pool)	2	13.0045	1.84x	91.8%
Multiprocess (Pool)	3	12.5066	1.91x	63.7%
Multiprocess (Pool)	4	12.1235	1.97x	49.3%

6. Resultados

Los resultados demuestran cómo el volumen de datos y la complejidad matemática determinan la viabilidad y efectividad de las distintas estrategias de concurrencia y paralelismo en Python.

En el escenario A

La ejecución Secuencial (Base) dominó de forma absoluta este escenario, completando la tarea en apenas **0.000043 segundos**. Las implementaciones con hilos (Threads/Multithreads) resultaron más lentas, arrojando *speedups* negativos (0.06x a 0.43x). Por su parte, las arquitecturas de procesos (Process/Multiprocess) registraron un rendimiento catastrófico, tardando alrededor de 0.1 segundos (magnitudes de

tiempo inmensamente mayores) y marcando una eficiencia del 0.0%.

Este escenario expone la penalización de la burocracia computacional. Cuando la tarea matemática es trivial (calcular 8 factoriales pequeños toma microsegundos en la CPU), el costo de preparación supera ampliamente al costo de ejecución.

En los procesos, el sistema operativo debe clonar el intérprete de Python, reservar nuevos espacios en memoria y utilizar la Comunicación Interprocesos (IPC) para enviar y recibir los datos serializados. Este esfuerzo inicial (*overhead*) hace que la estrategia paralela sea contraproducente.

En los Hilos, aunque comparten memoria y no sufren la penalización del IPC, el simple costo de instanciarlos y gestionar el cambio de contexto sigue siendo mayor que resolver la lista de forma lineal.

En el escenario B

Al inyectar una carga fuertemente *CPU-bound*, el tiempo Secuencial se disparó a 23.88 segundos. En este entorno, las estrategias de Hilos fracasaron en escalar el rendimiento, manteniéndose atascadas en aprox 22.6 segundos sin importar cuántos *workers* se añadieran, con la eficiencia desplomándose del 105% al 26%. Por el contrario, el Multiprocesamiento (sería el pool) logró el mejor rendimiento con 2 *workers*, reduciendo el tiempo a 13.00 segundos, logrando un *speedup* de 1.84x y una alta eficiencia del 91.8%. Añadir 3 o 4 procesos apenas redujo el tiempo (12.12 s), arruinando la eficiencia (<50%).

A pesar de declarar 4 *workers* en Threads, el tiempo no mejoró. Esto se debe al *Global Interpreter Lock* (GIL) de CPython, un mecanismo de seguridad que impide que múltiples hilos ejecuten instrucciones de Python simultáneamente en distintos núcleos. Los hilos terminaron ejecutándose de forma concurrente (turnándose), no paralela, ahogándose mutuamente al competir por el candado del GIL.

Los procesos aislados esquivan el GIL al tener cada uno su propio intérprete y espacio de memoria. El rendimiento alcanzó su "punto dulce" exacto en 2 *workers* (91.8% de eficiencia) porque el procesador físico de la prueba tiene exactamente 2 Cores. Al intentar forzar 3 o 4 procesos, el sistema operativo tuvo que empezar a multiplexar 4 cargas pesadas en solo 2 núcleos físicos, lo que reintroduce fricción por cambios de contexto y explica la caída en picada de la eficiencia al 49.3%.

7. Conclusión

El presente experimento logró medir y contrastar empíricamente el rendimiento del paralelismo y la concurrencia en Python, demostrando que la optimización del tiempo de ejecución no depende de aplicar ciegamente la mayor cantidad de recursos posibles, sino de analizar la naturaleza de la carga de trabajo frente a los límites físicos del hardware y las restricciones internas del propio lenguaje.

Para cargas de cómputo ligero y bajo volumen de datos (Escenario A): La técnica más eficiente fue la Ejecución Secuencial. Quedó evidenciado que cuando la tarea matemática es trivial, la penalización impuesta por el *overhead* (el costo de instanciación en memoria y la Comunicación Inter-Procesos) destruye el rendimiento. En estos casos, la burocracia de preparar el paralelismo tarda magnitudes de tiempo mayores que simplemente resolver el problema de forma lineal.

Para cargas de cómputo pesado y alto volumen de datos (Escenario B): La técnica más eficiente fue el Multiprocesamiento (Process/Pool) utilizando exactamente 2 Workers. Esta arquitectura fue la única capaz de evadir el bloqueo del GIL que asfixiaba a los hilos. Al emparejar matemáticamente la cantidad de *workers* con la cantidad de núcleos físicos reales del equipo (2 Cores), se logró una distribución perfecta de la carga de trabajo, obteniendo el mejor tiempo de ejecución con un Speed-up de 1.84x y maximizando la eficiencia de la máquina al 91.8%.